ZAINA RAHEEN A.P
241801328

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans

# 1. EXPLORE & PREPROCESS DATA

np.random.seed(42)
n_samples = 200
sqft = np.random.normal(2000, 500, n_samples)
bedrooms = np.random.randint(1, 6, n_samples)
price = 150 * sqft + 10000 * bedrooms + 0.03 * (sqft**2) + np.random.normal(0, 15000, n_samples)
df = pd.DataFrame({'SqFt': sqft, 'Bedrooms': bedrooms, 'Price': price})
print(df.head())
```

```
          SqFt  Bedrooms          Price
0  2248.357077         2  526980.691607
1  1930.867849         2  403527.789931
2  2323.844269         1  532144.380904
3  2761.514928         1  663012.021743
4  1882.923313         1  388313.304263
```

2.TRAIN REGRESSION MODELS

```
# Features and Target
X = df[['SqFt', 'Bedrooms']]
y = df['Price']

# Split data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Linear Regression

```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
print(f"Linear Regression Score: {lr.score(X_test_scaled, y_test):.2f}")
```

```
Linear Regression Score: 0.98
```

Polynomial Regression (Degree 2)

```
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)
poly_reg = LinearRegression()
poly_reg.fit(X_train_poly, y_train)
y_pred_poly = poly_reg.predict(X_test_poly)
print(f"Polynomial Regression Score: {poly_reg.score(X_test_poly, y_test):.2f}")
```

```
Polynomial Regression Score: 0.99
```

Support Vector Regression (SVR)

```
svr = SVR(kernel='rbf', C=10000, epsilon=1000)
svr.fit(X_train_scaled, y_train)
y_pred_svr = svr.predict(X_test_scaled)
print(f"SVR Score: {svr.score(X_test_scaled, y_test):.2f}")
```

```
SVR Score: 0.80
```

3. EVALUATE MODELS (MAE, MSE, RMSE)

```
def print_metrics(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    print(f"--- {name} ---")
    print(f"MAE  : ${mae:,.2f}")
    print(f"MSE  : {mse:,.2f}")
    print(f"RMSE : ${rmse:,.2f}\n")

print_metrics("Linear Regression", y_test, y_pred_lr)
print_metrics("Polynomial Regression", y_test, y_pred_poly)
print_metrics("SVR", y_test, y_pred_svr)
```

```
--- Linear Regression ---
MAE  : $14,664.37
MSE  : 300,735,835.65
RMSE : $17,341.74

--- Polynomial Regression ---
MAE  : $11,951.06
MSE  : 197,849,423.09
RMSE : $14,065.90

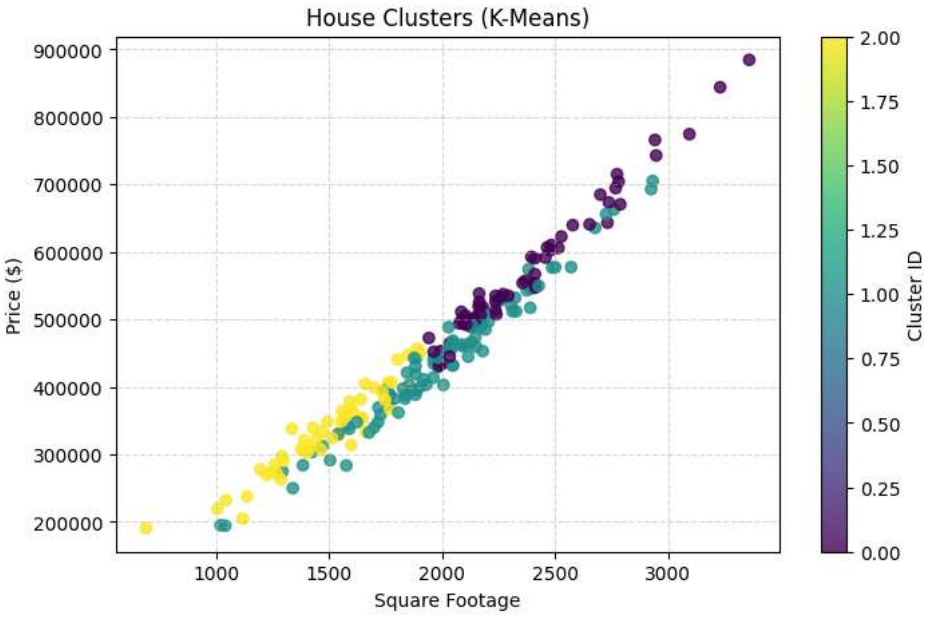
--- SVR ---
MAE  : $32,053.86
MSE  : 2,950,800,271.43
RMSE : $54,321.27
```

K-Means Clustering

```
X_scaled = scaler.fit_transform(X)

# Group into 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# Plot clusters
plt.figure(figsize=(8, 5))
plt.scatter(df['SqFt'], df['Price'], c=df['Cluster'], cmap='viridis', alpha=0.8)
plt.title('House Clusters (K-Means)')
plt.xlabel('Square Footage')
plt.ylabel('Price ($)')
plt.colorbar(label='Cluster ID')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```


House Clusters (K-Means)
Price ($)
Square Footage
Cluster ID
900000
800000
700000
600000
500000
400000
300000
200000
1000
1500
2000
2500
3000
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00